Does each paper answer our question?

Only one paper (Graetsch et al.) compares engineers and managers directly. Two others look at the question from the manager side only. The remaining three don't really address it.

#	Paper	Answers the question?	Roles covered	Reason
1	Graetsch et al. (2023), Dealing with Data Challenges	Yes	Both	Interviews engineers, data experts, team managers, and executives about the same problems and reports how each group dealt with them.
2	Suwarno et al. (2025), ML for Developer Productivity	Yes (one-sided)	Managers about engineers	Shows how 12 managers define and judge engineer productivity, but engineers were never asked.
3	Nguyen (2016), Workplace Factors in Agile Teams	Yes (one-sided)	Managers about engineers	10 managers say their job is to support the team and let engineers decide how the work gets done. No engineers took part.
4	Babu (2025), Parental Leadership	No	Two manager roles	Compares product managers with engineering managers. Engineers aren't part of the comparison, and there is no data.
5	Bahi et al. (2024), GenAI for Agile	No	Neither directly	Looks at where GenAI could fit into agile work. Roles come up only briefly.

#	Paper	Answers the question?	Roles covered	Reason
6	Zhang et al. (2026), LLMs for SE survey	No	Neither	Reviews research on AI for coding tasks. It looks at papers, not people.

1. Dealing with Data Challenges when Delivering Data-Intensive Software Solutions (keep)

Authors, year, where published: Ulrike M. Graetsch, Hourieh Khalajzadeh, Mojtaba Shahin, Rashina Hoda, John Grundy; 2023. The uploaded copy is arXiv v2 (March 2023), headed "Submitted to IEEE Transactions on Software Engineering". Check the published TSE version before citing.
Type of study: Interviews analysed with socio-technical grounded theory.
Who they studied: Both. 24 practitioners in multi-disciplinary teams, including software engineers, data scientists and analysts, domain experts, team managers and executives (CIO, Director of Data). 22 of 24 were in Australia or New Zealand.
Main finding: Most of the trouble these teams described came from the data itself. People struggled to get access, to understand what fields meant, to match up data from different sources, and to fix bad records. The causes they gave were weak data literacy, missing documentation, and waiting on other teams. Leaders were responding with data governance, engineers with monitoring tools, and team leads by working on communication and cross-training.
Engineer vs manager: Each role saw the same data problems from its own position. Executives talked about governance and long-term data strategy. Engineers dealt with the hands-on side, such as firewalls, access requests, and building monitoring tools, while team leaders talked mostly about culture and communication. Several participants described technical staff as introverted and protective of their craft, and business staff as more outgoing. One CIO said he worried less about technical detail once he had the right people (P14, p. 13).
Useful for us because: It's the only paper we kept that interviewed both engineers and managers, so we can compare the roles directly. Fig. 3 (p. 8) and Fig. 4 (p. 15) could also be reused in the video.
Quote worth citing: A development manager on engineering: "Software engineering is my craft. So I don't compromise much." (P13, p. 8).

2. Exploring Machine Learning to Support Software Managers' Insights on Software Developer Productivity (keep)

Authors, year, where published: Suwarno, Yefta Christian, Keaton Yoputra, Yuki Estrada; 2025; Journal of Applied Data Sciences, 6(3), pp. 1504–1524.
Type of study: Mixed: semi-structured interviews, then building and testing ML classifiers.
Who they studied: Managers only. 12 software managers (team leads, product and program managers, a head of tech) from six Indonesian software houses. Screenshots and URLs came

from developers at one partner company, but developers were not interviewed.
Main finding: Most managers said a productive developer finishes tasks on time without cutting corners on quality. Some also mentioned adaptability, teamwork, and knowing what stakeholders want. Distraction came up often, and managers found status reports and check-ins slow and biased. The authors then built screenshot and URL classifiers (99.6% and 88.5% accuracy) to estimate how much of a developer's time goes to actual work.
Engineer vs manager: We only hear the manager side here. Managers want an objective view of what developers are doing, though they also admit developers need some freedom (p. 1511). All 12 managers liked the proposed tool (p. 1520), but nobody asked the developers who would be monitored. We think that missing view matters for our project.
Useful for us because: It gives us a sourced manager definition of productivity. It also raises the question of how monitoring fits with engineer autonomy, which we can ask about in interviews.
Number worth citing: 11 of 12 managers defined productivity as on-time completion without sacrificing quality; only p1 differed (p. 1509).

3. Workplace Factors that Shape Agile Software Development Team Project Success (keep, with caution)

Authors, year, where published: Dan Schilling Nguyen (Walden University); 2016; American Scientific Research Journal for Engineering, Technology, and Sciences (ASRJETS), 17(1), pp. 323–391.
Type of study: Qualitative phenomenological study using an open-ended email questionnaire (nine questions).
Who they studied: Managers only. 10 agile project managers or team leaders at US-based global workplaces; 7 in IT, 2 in manufacturing, 1 in defense.
Main finding: Managers linked agile success to team culture, building trust early and sharing knowledge openly, for example through a team wiki. They want team members with broad skills who can manage themselves. Asked about leadership style, most chose servant leadership, meaning they support the team rather than direct it.
Engineer vs manager: The managers split the work this way: engineers decide how and when the work gets done, and managers remove blockers and keep everyone working toward the same goal. 8 of 10 agreed the experts should make those calls (p. 348).
Useful for us because: Of our papers, this one puts the "manager as supporter" idea most directly. In interviews, we can check whether engineers see their managers this way.
Caution: The journal is not well known, and the paper reads like a cut-down dissertation. Almost every theme is reported at exactly 9 of 10 participants, which looks odd. We'd use it as supporting evidence only.
Quote worth citing: A participant described the ideal: "Leaders should enable teams rather than direct them." (Participant 1, p. 348). 9 of 10 managers chose servant leadership (p. 348).

4. Parental Leadership in Product-Engineering Partnerships for Managing Software Teams (no)

Authors, year, where published: Aishwarya Babu; 2025; International Journal on Science and Technology (IJSAT), 16(2), 4 pages.

Type of study: Conceptual framework (opinion); no data collected.
Who they studied: No one. It describes product managers (PMs) and engineering managers (EMs).
Main finding: The author suggests the PM and EM lead a team together like "co-parents". The PM handles outward-facing work such as customers and the roadmap, and the EM looks after quality and team health.
Engineer vs manager: It compares two kinds of manager, not managers with engineers. Engineers only show up as the team being looked after.
Why excluded: There's no evidence behind the claims, the journal's peer review is unclear, and some references don't match what they're cited for. For example, a paper on the Personal Software Process is cited for the Team Software Process (ref. 3).
Worth keeping from it: Its first reference, Kalliamvakou et al. (2018) on what makes a great software manager, looks worth finding. The speed-versus-quality split would also make a good interview question.
Quote: "PMs may push for speed, EMs for quality." (p. 3).

5. Integrating Generative AI for Advancing Agile Software Development and Mitigating Project Management Challenges (no)

Authors, year, where published: Anas Bahi, Jihane Gharib, Youssef Gahi; 2024; International Journal of Advanced Computer Science and Applications (IJACSA), 15(3), pp. 54–61.
Type of study: Literature-based conceptual framework. It promises a real case study but does not present one.
Who they studied: No participants.
Main finding: It lists ten common agile problems, such as unclear requirements, unclear roles, resistance to change, and poor estimates, and suggests how GenAI could help with each. It then shows where GenAI could fit into each Scrum stage. The authors note that the same prompt gives different answers on different runs, so people still need to check the output.
Engineer vs manager: It touches the question only briefly. The GenAI uses it lists fall into engineer tasks (code, tests, reviews) and manager tasks (estimates, stakeholder updates, role descriptions). It says the programmer role will change but offers no evidence.
Why excluded: There's no data, and the roles question gets only a few lines. Most of its pain-point table comes from one master's thesis (ref. 8).
Worth keeping from it: It cites Hoda and Murugesan (2016) on agile project management challenges, which could work as a replacement.
Quote: "There will be changes to several established roles, like programmer." (p. 60).

6. A Survey on Large Language Models for Software Engineering (no)

Authors, year, where published: Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Shengcheng Yu, Weisong Sun, Yun Yang, Zhenyu Chen; 2026; Science China Information Sciences, 69(4), article 141102 (52 pages).
Type of study: Systematic literature review of research papers up to August 2024.
Who they studied: Research papers, not people: 988 studies, including 926 that apply LLMs to 112 software engineering tasks.

Main finding: Most LLM research targets development, testing, and maintenance. Very few studies look at management, requirements, or design, and the authors list this as a gap.
Engineer vs manager: Not directly. The research covers the engineer's job much more than the manager's. The few management studies looked at effort estimation, developer emotions, and mining repositories for team productivity data.
Why excluded: It's about AI techniques, not how engineers and managers differ, and the rest of the 52 pages adds little to our question.
Number worth citing: Only 7 of 926 LLM studies deal with software management tasks (Table 7, p. 141102:16). We could use this if we want to argue that AI tools so far mostly serve engineers.

Interview question ideas

Each question checks a claim that came from only one side, or that had no industry evidence behind it.

1. How do you define a productive day? (Ask engineers and managers the same question; Suwarno only asked managers.)
2. How would you feel about tools that take screenshots to measure focus time? Would it change trust in your team? (Suwarno vs Nguyen.)
3. Does your manager mostly clear the way, or mostly direct the work? Does that change near a deadline? (Nguyen's servant leadership claim.)
4. When speed and quality clash, who pushes for which, and who makes the final call? (Babu's PM-speed, EM-quality split.)
5. Is the "quiet engineer, talkative business person" picture true where you work? Does it cause problems? (Graetsch, p. 8.)
6. When a dependency on another team or data source blocks you, who chases it: you or your manager? (Graetsch, inter-team dependencies.)
7. How has AI coding help changed your work? Do managers use AI for estimates or reports? (Zhang and Bahi.)
8. For people who moved from engineer to manager: what did you have to stop doing, and what was hardest to learn? (Links to Kalliamvakou et al.)